

# SIMATS ENGINEERING

## Co3-ASSESSMENT TOOL 2 Case study

COURSE NAME: Computer Networks

COURSE CODE: CSA07

---

### COURSE OUTCOME COVERED

**CO3:** *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

Dr.V.Parthipan

**QUESTIONS:4**

**Total Marks:40**

---

### Code of Conduct:

*I **N. Hemasree (192511100)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student: N. Hemasree**

| Criteria | Understanding the case<br>2.5 | Application of Concepts<br>2.5 | Analysis<br>2 | Solution Design<br>2 | Clarity<br>1 | Total<br>(10) |
|----------|-------------------------------|--------------------------------|---------------|----------------------|--------------|---------------|
| Q1       |                               |                                |               |                      |              |               |
| Q2       |                               |                                |               |                      |              |               |
| Q3       |                               |                                |               |                      |              |               |
| Q4       |                               |                                |               |                      |              |               |

**QUESTIONS:4****Total Marks:40**

1. A multinational IT company uses a cloud-based video conferencing platform for daily client meetings. During peak business hours, employees experience frequent voice distortion, frozen video frames, and delayed communication. Network monitoring reports indicate an **8% packet loss, 180 ms jitter**, and fluctuating bandwidth utilization. The IT administrator suspects that the issue is related to the transport protocol or application-level buffering strategy.

## Questions

1. Analyze whether the problem is primarily caused by the **transport protocol (TCP/UDP)** or the **application layer**.
2. Justify your answer based on the communication requirements of real-time multimedia applications.
3. Recommend suitable protocol-level or application-level improvements to enhance conferencing quality.

2. A multinational software company hosts its web services across multiple continents. Employees frequently report long delays before the application homepage loads. Network analysis shows that the **average DNS lookup time is approximately 800 ms**, significantly affecting user experience. The organization plans to redesign its DNS infrastructure while ensuring continuous service availability during server failures.

## Questions

1. Analyze the reasons behind the high DNS resolution time.
2. Propose a suitable DNS architecture using techniques such as **Anycast DNS, DNS pre-fetching**, and **TTL optimization** to reduce the lookup time below **50 ms**.
3. Evaluate the advantages and limitations of your proposed architecture in terms of performance, scalability, and availability.

3. A cloud service provider delivers applications to millions of users worldwide. During promotional events, users experience intermittent delays in accessing cloud-hosted services. Investigation reveals that DNS queries are taking nearly **800 ms** because all requests are directed to a single primary DNS server. The company wants a highly available DNS solution capable of handling global traffic

efficiently.

### Questions

4. Examine the impact of centralized DNS architecture on application performance.
5. Design a distributed DNS solution that minimizes lookup latency while maintaining high availability.
6. Analyze how **TTL values**, **Anycast routing**, and **DNS caching** influence system performance and fault tolerance.

4.A financial institution operates an electronic stock trading platform where thousands of buy and sell orders are submitted every second. During market opening hours, the **average order acknowledgment latency increases from 1 ms to 40 ms**, resulting in delayed trade confirmations and customer dissatisfaction. Network engineers observe increased TCP retransmissions, longer connection establishment times, and excessive buffering.

### Questions

1. Analyze three possible TCP-related factors responsible for the latency increase, considering **flow control**, **Nagle's algorithm**, and **SYN queue limitations**.
2. Explain how each factor affects transaction processing in high-frequency systems
3. Recommend appropriate TCP configuration changes to reduce latency and improve transaction reliability.

### ANSWERS:

#### 1.Cloud-Based Video Conferencing Network Issue

##### Understanding of Case

The multinational IT company is facing voice distortion, frozen video frames, and delayed communication during peak business hours. The network monitoring data shows:

- **Packet loss:** 8%
- **Jitter:** 180 ms
- **Bandwidth:** Fluctuating during peak hours
- **Application:** Real-time cloud-based video conferencing

These values indicate that the conferencing application is experiencing unstable network conditions. Real-time audio and video require low delay, low jitter, sufficient bandwidth, and minimal packet loss.

The main question is whether the problem is caused by TCP/UDP transport protocol behavior or by the application's buffering strategy.

## Application of Concepts

### TCP vs UDP for Video Conferencing

| Feature                            | TCP                          | UDP                      |
|------------------------------------|------------------------------|--------------------------|
| Connection                         | Connection-oriented          | Connectionless           |
| Reliability                        | High                         | Low                      |
| Retransmission                     | Yes                          | No                       |
| Delay                              | Higher due to retransmission | Lower                    |
| Packet loss handling               | Retransmits lost packets     | Application handles loss |
| Suitable for real-time audio/video | Less suitable                | Highly suitable          |

TCP guarantees reliable delivery by retransmitting lost packets. However, retransmissions can introduce additional delay and **head-of-line blocking**, where later packets wait for a missing packet.

UDP does not retransmit packets automatically. This makes it more appropriate for real-time conferencing because receiving slightly incomplete audio/video on time is generally better than receiving perfect data late.

Modern conferencing applications commonly use UDP-based real-time media protocols, while application-level mechanisms handle packet loss, jitter, congestion, and quality adaptation.

## Analysis

### Is the problem primarily TCP/UDP or the application layer?

The problem is primarily a network-quality and application-level adaptation issue rather than simply a TCP-vs-UDP problem.

The important indicators are:

### **8% Packet Loss**

An 8% packet loss rate is very high for a video conference. Lost packets can cause:

- Missing audio segments
- Distorted voice
- Frozen video
- Pixelation
- Frame skipping

With UDP, lost packets are normally not retransmitted, so the application must compensate for them.

### **180 ms Jitter**

Jitter represents variation in packet arrival time. 180 ms is extremely high for interactive communication.

For example:

Normal packet arrival:

P1 ---- P2 ---- P3 ---- P4

High jitter:

P1 ----- P2 -- P3 ----- P4

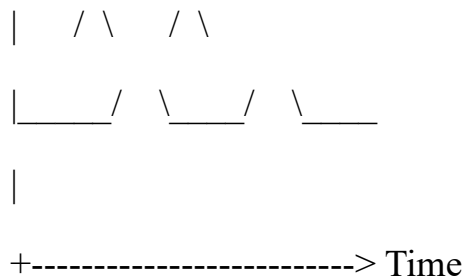
The receiving application cannot immediately play packets because they may arrive at unpredictable times.

### **Fluctuating Bandwidth**

During peak business hours, bandwidth may become congested.

Available Bandwidth

|  
High |    ^    ^



When available bandwidth suddenly decreases, video applications may continue sending data at a higher rate, causing congestion, packet loss, and increased delay.

### **Role of Application-Level Buffering**

The application uses a **jitter buffer** to temporarily store packets before playing them.

#### **Small buffer**

Packets → [Small Buffer] → Audio/Video

Advantages:

- Low delay

Disadvantages:

- More likely to experience interruptions when jitter is high

#### **Large buffer**

Packets → [Large Buffer] → Audio/Video

Advantages:

- Handles jitter better

Disadvantages:

- Introduces additional latency

Therefore, simply increasing the buffer size is **not a complete solution**. A very large buffer could make conversations noticeably delayed.

The conferencing application should use an **adaptive jitter buffer** that dynamically adjusts according to current network conditions.

## **Recommended Solution Design**

### **A. Use UDP for Real-Time Media**

For interactive voice and video, use UDP-based media transport rather than relying on TCP for the media stream.

UDP avoids unnecessary retransmission delays.

Camera/Microphone



Video/Audio



UDP-based Media



Internet



Adaptive Receiver



Speaker/Display

### **B. Adaptive Bitrate**

The application should continuously monitor:

- Packet loss
- Jitter
- Available bandwidth
- Round-trip time

When network quality decreases:

Good Network



High Video Quality

1080p / High Bitrate

Poor Network



Reduce Bitrate



720p → 480p



Maintain Communication

Instead of allowing packets to be lost excessively, the application should reduce the video bitrate and resolution.

### **C. Adaptive Jitter Buffer**

The application should dynamically change its buffer size.

Low Jitter



Small Buffer



Low Latency



High Jitter



Larger Buffer



Smoother Playback

This provides a balance between **smooth playback and communication delay**.

#### **D. Packet Loss Concealment**

For audio, the application can use **Packet Loss Concealment (PLC)**.

If a small portion of speech is lost:

Received:

"Good mor--- everyone"

PLC:

"Good morning everyone"

The system estimates the missing audio instead of waiting for retransmission.

#### **E. Forward Error Correction**

Forward Error Correction (FEC) can add redundant information to packets.

Original packets:

P1 P2 P3 P4

Redundant information:

FEC

If P3 is lost



FEC helps reconstruct P3

This is useful because the receiver can recover some lost information without requesting retransmission.

### **F. Quality of Service (QoS)**

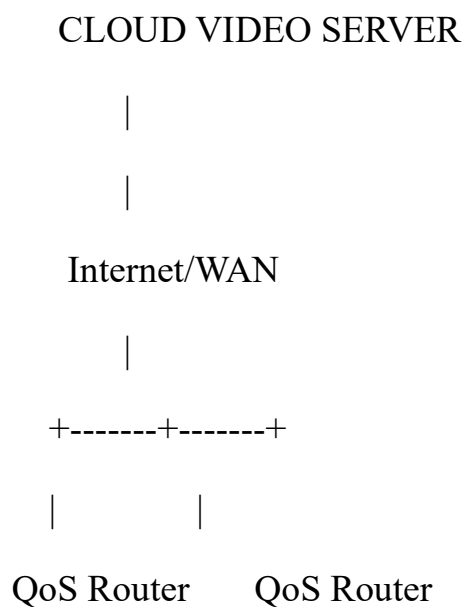
The company should prioritize real-time conferencing traffic over less timesensitive applications.

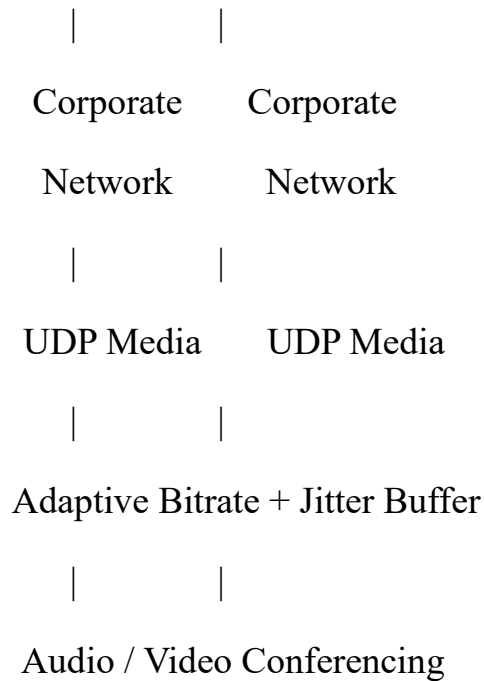
For example:

| <b>Traffic</b>        | <b>Priority</b> |
|-----------------------|-----------------|
| Voice                 | Very High       |
| Video                 | High            |
| Business applications | Medium          |
| File downloads        | Low             |
| Software updates      | Very Low        |

QoS can reduce the effect of congestion during peak business hours.

### **Overall Recommended Architecture**





The system should combine UDP-based real-time transport + adaptive bitrate + adaptive jitter buffering + packet-loss concealment/FEC + QoS.

## **Conclusion**

The conferencing problem is not caused solely by choosing TCP or UDP. The measured 8% packet loss, 180 ms jitter, and fluctuating bandwidth indicate significant network congestion and instability. However, the application's transport and buffering strategy determines how severely users experience these problems.

For real-time multimedia, UDP is generally more appropriate than TCP because timely delivery is more important than perfect delivery. The best solution is to use UDP-based real-time media with adaptive bitrate control, adaptive jitter buffering, FEC/packet-loss concealment, and QoS prioritization.

## **Final Recommendation**

Use UDP for real-time audio/video, implement adaptive jitter buffering and bitrate control, apply packet-loss recovery techniques such as FEC/PLC, and prioritize conferencing traffic using QoS.

This combination can significantly reduce voice distortion, frozen frames, and communication delays while maintaining acceptable conferencing quality.

## 2.Global DNS Infrastructure Redesign

### Understanding of the Case

The multinational software company hosts web services across multiple continents, but employees experience long delays while loading the application homepage.

The main problem identified is:

- Average DNS lookup time = 800 ms • Target DNS lookup time = below 50 ms
- Services are distributed globally.
- The DNS infrastructure must remain available even if a DNS server fails. An 800 ms DNS lookup time is very high for a web application and can significantly increase the time before the browser can establish a connection to the web server.

### Analysis

Reasons for High DNS Resolution Time

Several factors can contribute to the 800 ms DNS lookup time.

#### **A. Geographical Distance**

If users in India are querying a DNS server located in North America or Europe, the DNS request has to travel a long distance.

User in India

|  
| DNS Query



DNS Server in USA

|  
| DNS Response



User in India

This increases network latency.

#### **B. Multiple DNS Queries**

If the requested domain is not available in the local resolver's cache, the resolver may need to contact several DNS servers:

Client



Recursive Resolver



Root DNS



TLD DNS



Authoritative DNS



Response

Multiple round trips increase resolution time.

### C. Poor DNS Caching

If DNS records have a very short TTL (Time To Live), cached records expire frequently. Consequently:

Short TTL



Cache expires quickly



New DNS lookup



More network requests



Higher latency

### D. DNS Server Overload

If too many users depend on a small number of DNS servers, the servers can become overloaded, increasing response time.

### E. DNS Server Failure

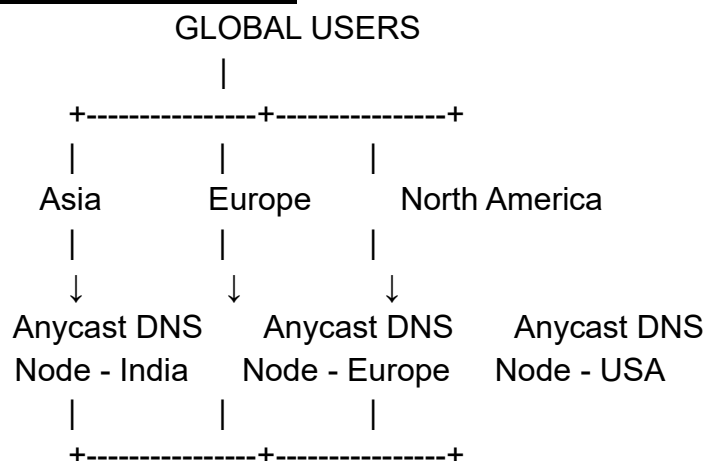
If one DNS server fails and there is no effective redundancy, queries may have to wait for timeout and retry another server.

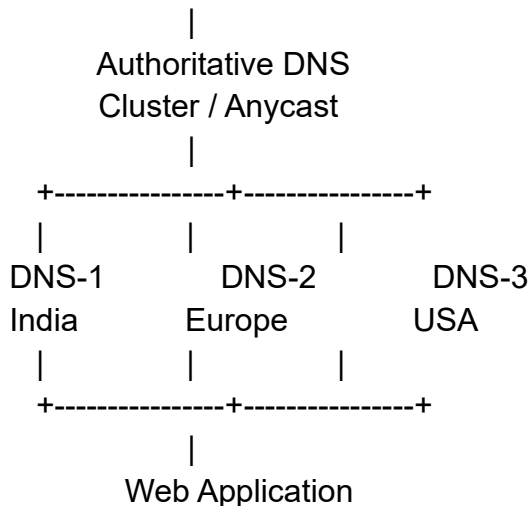
### Proposed DNS Architecture

The recommended architecture combines:

1. Anycast DNS
2. DNS caching
3. DNS pre-fetching
4. TTL optimization
5. Multiple geographically distributed authoritative DNS servers
6. Health monitoring and automatic failover

### Proposed Architecture





With Anycast DNS, multiple DNS servers advertise the same IP address. Routing directs the user's query to a nearby healthy DNS node.

### **Anycast DNS**

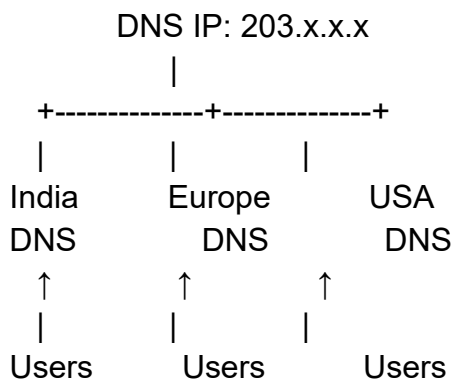
Anycast is the most important component of the proposed architecture.

How it works

Suppose the company has DNS servers in:

- India
- Singapore
- Europe
- USA

All servers advertise the same DNS service IP.



A user is normally routed to a nearby available DNS node.

### **Benefit** Instead

of:

India User → USA DNS the system can provide:

India User → India DNS

This significantly reduces network round-trip time.

### **DNS Pre-Fetching**

DNS pre-fetching allows DNS information to be resolved before it is actually needed.

For example, if the homepage is likely to request: `www.company.com`

`api.company.com` `images.company.com`

the required DNS records can be cached or prefetched before the corresponding resource is requested.

### **Process**

User requests homepage



DNS records already cached



No fresh DNS lookup required



Website loads faster

DNS prefetching is particularly useful for frequently accessed domains and resources.

### **TTL Optimization**

TTL determines how long a DNS record can remain in a resolver's cache.

A balanced TTL strategy should be used.

Very Short TTL

TTL = Very Short



Frequent DNS queries



Higher DNS load



Higher latency

Very Long TTL

TTL = Very Long



Excellent caching



But changes take longer to propagate

Recommended Strategy

Use moderate/optimized TTL values based on how frequently records change. For example:

| <b>DNS Record</b> | <b>Suggested Strategy</b> |
|-------------------|---------------------------|
|-------------------|---------------------------|

|                           |            |
|---------------------------|------------|
| Stable application domain | Longer TTL |
|---------------------------|------------|

|                             |              |
|-----------------------------|--------------|
| Frequently changing service | Moderate TTL |
|-----------------------------|--------------|

|                          |                        |
|--------------------------|------------------------|
| During planned migration | Temporarily reduce TTL |
|--------------------------|------------------------|

|                 |                    |
|-----------------|--------------------|
| After migration | Increase TTL again |
|-----------------|--------------------|

This reduces unnecessary DNS lookups while still allowing changes to propagate within an acceptable time.

## **DNS Caching**

Caching should exist at multiple levels:

Browser Cache



Operating System Cache



Local/Corporate Resolver Cache



Anycast DNS



Authoritative DNS

If the requested DNS record is already cached, the resolver can immediately return the result.

Therefore:

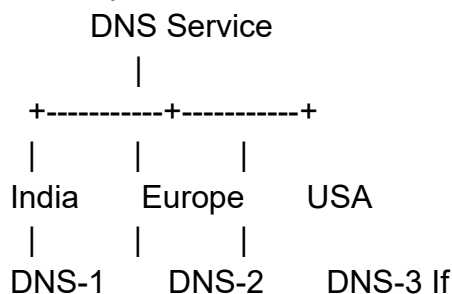
Cache hit → Low latency

Cache miss → Full DNS resolution

## **Failure Handling and Continuous Availability**

The company should deploy multiple DNS nodes across different geographical regions.

For example:



DNS-1 fails:

User



DNS-1



Routing changes



DNS-2 / DNS-3



Successful response

Health checks should continuously monitor the DNS nodes.

If a node becomes unavailable, its route can be withdrawn so that new queries are directed toward another healthy node.

## **Target Performance**

The current average lookup time is:

800 ms Target:



< 50 ms

The proposed improvements attack the main causes of delay:

| Technique          | Effect                                     |
|--------------------|--------------------------------------------|
| Anycast DNS        | Reduces geographical latency               |
| DNS caching        | Avoids repeated lookups                    |
| Technique          | Effect                                     |
| DNS pre-fetching   | Resolves records before they are needed    |
| TTL optimization   | Increases useful cache lifetime            |
| Multiple DNS nodes | Reduces server overload                    |
| Health monitoring  | Provides fast failure detection            |
| Load distribution  | Improves response time during peak traffic |

With geographically distributed DNS nodes and high cache-hit rates, sub-50 ms DNS response time is a realistic design target for users who are routed to a nearby DNS node, although the exact result depends on ISP, resolver, geography, and network conditions.

### **Evaluation of Proposed Architecture A. Performance**

#### **Advantages**

- Anycast reduces geographical distance.
- Caching reduces repeated DNS queries.
- Pre-fetching reduces lookup delay.
- TTL optimization improves cache efficiency.
- Load distribution prevents individual DNS servers from becoming bottlenecks.

#### **Limitation**

The company cannot guarantee <50 ms for every user because DNS latency also depends on the user's ISP, recursive resolver, Internet congestion, and geographic location.

#### **B. Scalability**

##### **Advantages**

The architecture is highly scalable.

New DNS nodes can be added in additional regions:

India → Singapore → Europe → USA → Australia

Traffic can be distributed across the global DNS infrastructure.

##### **Limitation**

Managing many globally distributed DNS nodes increases:

- Configuration complexity
- Monitoring requirements
- Operational cost

#### **C. Availability**

##### **Advantages**

There is no single DNS server on which the entire service depends. If one node fails:

DNS-1

↓  
DNS-2 / DNS-3

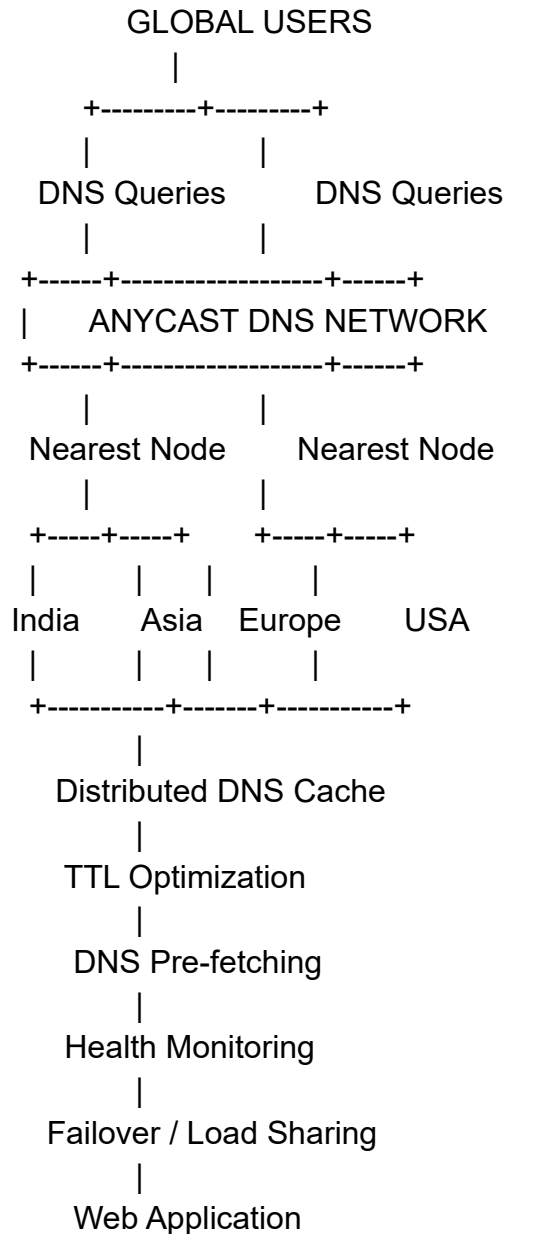
↓  
Service continues

This provides strong fault tolerance and continuous service availability.

Limitation

Anycast itself does not eliminate all failures. Routing problems, configuration errors, or a common failure affecting multiple DNS nodes can still affect availability.

### **Recommended Final Architecture**



### **Conclusion**

The 800 ms DNS lookup delay is mainly caused by factors such as geographical distance, cache misses, repeated recursive lookups, overloaded DNS servers, and inefficient TTL configuration.

The best solution is a globally distributed Anycast DNS architecture combined with DNS caching, pre-fetching, optimized TTL values, health monitoring, and redundant authoritative DNS servers.

#### Final Recommendation

Deploy Anycast DNS nodes across major geographic regions, maximize effective DNS caching through optimized TTL values, use DNS pre-fetching for frequently accessed resources, and maintain multiple redundant DNS servers with healthbased failover.

This architecture improves performance by reducing lookup latency, scalability by distributing DNS traffic globally, and availability by eliminating dependence on a single DNS server.

### 3. Distributed DNS Solution for Global Cloud Services

#### Understanding of Case

The cloud service provider serves millions of users worldwide. During promotional events, the number of DNS requests increases significantly.

The current architecture sends all DNS queries to one primary DNS server, resulting in:

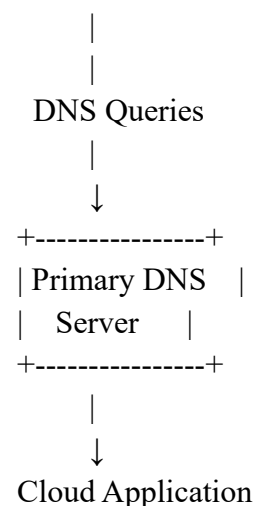
- DNS lookup latency of nearly 800 ms
- Increased response time
- Server overload
- Possible DNS timeouts
- Poor application performance
- Single point of failure

The main requirement is to replace the centralized DNS architecture with a distributed, highly available DNS infrastructure.

#### Impact of Centralized DNS Architecture

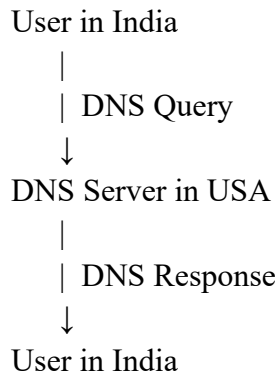
The existing architecture can be represented as:

Millions of Global Users



#### A. High Latency

Users located far away from the primary DNS server experience longer network round trips. For example:

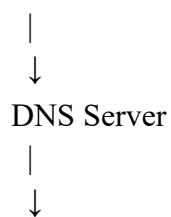


This geographical distance contributes to the 800 ms lookup delay.

### **B. Single Point of Failure**

If the primary DNS server fails:

Users



Application inaccessible

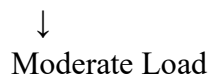
Even if the cloud application itself is running, users may be unable to find its IP address.

### **C. Server Overload** During

promotional events:

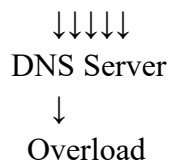
Normal Traffic:

Users → DNS Server



Peak Traffic:

Millions of Users



The server may experience increased CPU, memory, network utilization, and query processing delays.

### **D. Poor Scalability**

A single DNS server cannot efficiently handle rapidly increasing global traffic.

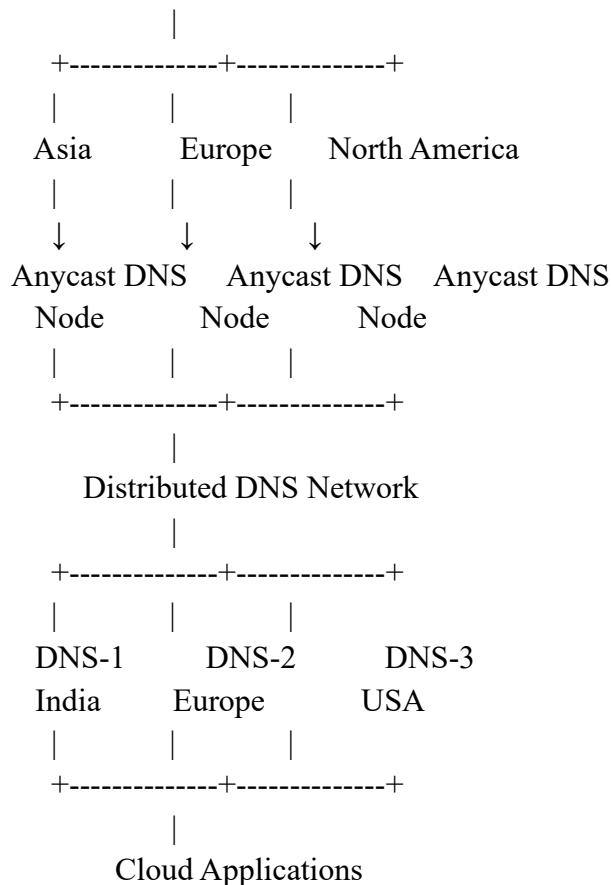
Therefore, centralized DNS creates both a performance bottleneck and availability risk.

### **Distributed DNS Architecture**

A suitable solution is a globally distributed Anycast DNS architecture with multiple DNS servers, DNS caching, optimized TTL values, health monitoring, and automatic failover.

Proposed Architecture

GLOBAL USERS



Instead of sending every query to one server, DNS requests are distributed among multiple geographically located DNS nodes.

### **Anycast Routing**

Anycast routing allows multiple DNS servers in different locations to advertise the same IP address.

The Internet routing system generally directs the user toward an appropriate/nearby available DNS node.

Example

User in India



India Anycast DNS



DNS Response

Instead of:

User in India



DNS Server in USA



DNS Response

This reduces network round-trip time.

During Server Failure

Suppose the India DNS node fails:

User



India DNS



Anycast routing



Another healthy DNS node



DNS Response

Thus, the service can continue operating without depending on one physical DNS server.

### **DNS Caching**

DNS caching stores recently resolved domain information so that repeated queries do not require a complete DNS resolution process.

Without Caching

User



Recursive Resolver



Root



TLD



Authoritative DNS



Response

This can require multiple network interactions.

With Caching

User



Cached DNS Record



Immediate Response

This greatly reduces lookup latency and decreases the load on authoritative DNS servers.

During Promotional Events

Caching becomes particularly valuable because millions of users may request the same domain.

Millions of Users



Local/Recursive DNS Cache



Cached IP Address



Cloud Application

Therefore, many queries can be answered without contacting the authoritative DNS infrastructure.

### **Effect of TTL Values**

TTL (Time To Live) determines how long a DNS record can remain in a resolver's cache.

Short TTL

Short TTL



Record expires quickly



More DNS queries



Higher DNS traffic



Higher server load

**Advantage:** DNS changes propagate relatively quickly.

**Disadvantage:** More frequent DNS lookups and increased load.

Long TTL

Long TTL



Record remains cached longer



Fewer DNS queries



Lower DNS load



Faster responses

**Advantage:** Better performance and lower DNS traffic.

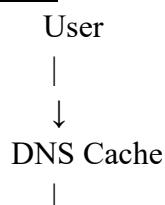
**Disadvantage:** Changes to DNS records take longer to propagate.

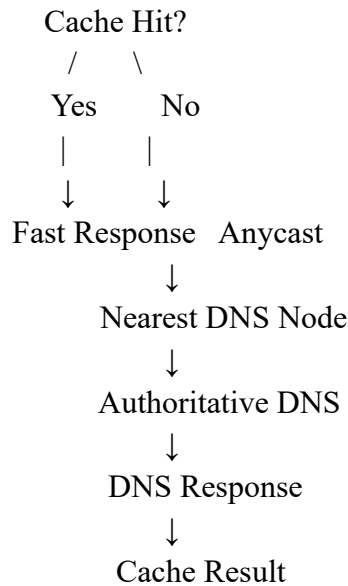
The provider should use balanced TTL values based on how frequently records change.

| <b>Situation</b>       | <b>TTL Strategy</b>     |
|------------------------|-------------------------|
| Stable cloud service   | Longer TTL              |
| Frequently changing IP | Moderate TTL            |
| Planned migration      | Temporarily shorter TTL |
| After migration        | Increase TTL again      |

TTL should not simply be set to the maximum possible value because operational changes and failover requirements must also be considered. **Combined Effect of TTL, Anycast**

**and Caching** These three techniques work together:





### **Performance Improvement**

- DNS caching reduces unnecessary queries.
- TTL optimization controls how long cached records remain valid.
- Anycast reduces the distance between users and DNS servers.

Together, these can significantly reduce DNS lookup latency compared with the centralized 800 ms design.

### **Fault Tolerance and High Availability**

The distributed architecture removes the single point of failure.

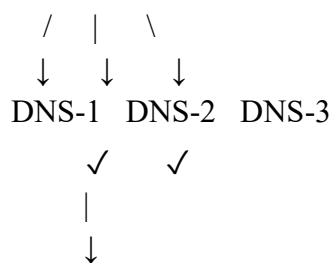
Centralized Architecture

DNS Server

Entire service  
becomes difficult to  
access

Distributed Architecture

DNS Network



Service continues

If one DNS node fails, other nodes continue answering requests.

Health monitoring should continuously check DNS nodes. Failed nodes can be removed from service/routing until they recover.

### **Performance, Scalability and Availability Analysis**

| Feature | Performance | Scalability | Fault Tolerance |
|---------|-------------|-------------|-----------------|
|---------|-------------|-------------|-----------------|



|                      |                           |                      |           |
|----------------------|---------------------------|----------------------|-----------|
| Centralized DNS      | Poor during peak load     | Low                  | Low       |
| DNS Caching          | Very high improvement     | Reduces DNS load     | Good      |
| Optimized TTL        | Improves cache efficiency | Reduces query volume | Moderate  |
| Anycast DNS          | Low latency               | Excellent            | Excellent |
| Multiple DNS servers | High                      | Excellent            | Excellent |

**Advantages of Proposed Solution**

### 1. Lower Latency

Users are directed toward appropriate geographically distributed DNS nodes.

### 2. High Availability

Failure of one DNS server does not bring down the entire DNS service.

### 3. Better Scalability

Traffic is distributed across multiple DNS nodes.

### 4. Reduced DNS Load

Caching prevents repeated queries from reaching authoritative servers.

### 5. Better Performance During Promotions

Large numbers of repeated requests can be served from caches, while Anycast distributes new queries across the DNS infrastructure.

### 6. Fault Isolation

A failure in one geographic region does not necessarily affect users in other regions.

## **Limitations**

### **Anycast**

- Requires proper global routing configuration.
- Routing problems can affect reachability.
- Does not protect against a common configuration error affecting all nodes.

### **DNS Caching**

- Cached records can become stale.
- Incorrectly configured TTLs can delay DNS changes.

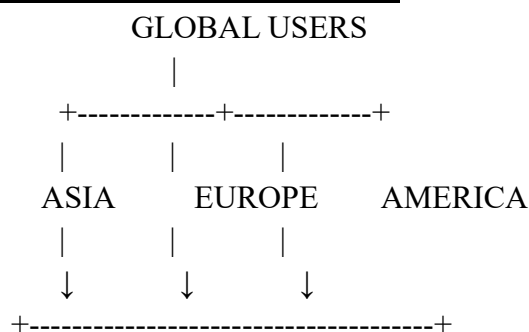
### **TTL**

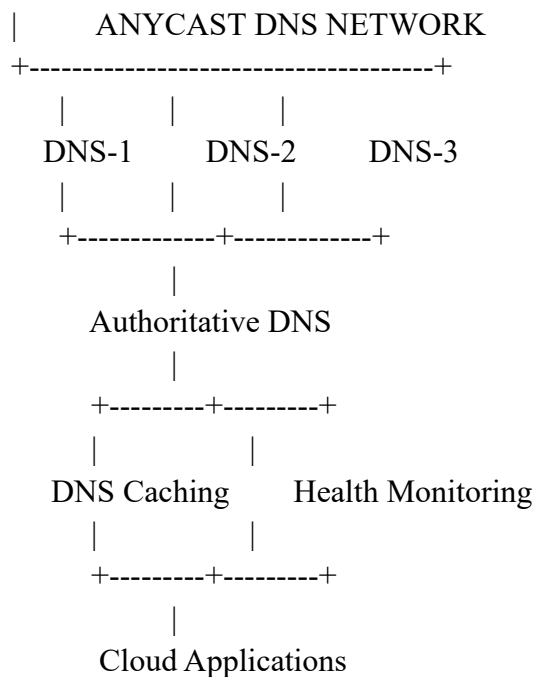
- Very short TTL increases DNS traffic.
- Very long TTL can delay changes and failover information from reaching users.

### **Distributed Infrastructure**

- More expensive than a single DNS server.
- Requires continuous monitoring and configuration management.
- More complex to operate.

## **Final Recommended Architecture**





### **Conclusion**

The 800 ms DNS delay is primarily a consequence of the centralized architecture, where millions of global users depend on a single DNS server. This creates high latency, server overload, poor scalability, and a single point of failure.

The recommended solution is a distributed Anycast DNS architecture with multiple geographically distributed DNS nodes, DNS caching, optimized TTL values, and healthbased failover.

Anycast reduces lookup latency, DNS caching reduces repeated queries, and optimized TTL values balance performance with the need for timely DNS updates. Together, these mechanisms provide a highly scalable, high-performance, and fault-tolerant DNS infrastructure suitable for millions of global users.

## **4.TCP Latency in an Electronic Stock Trading Platform**

### **Understanding of the Case**

The financial institution operates a **high-frequency electronic stock trading platform** where thousands of buy and sell orders are processed every second.

During normal operation:

- Order acknowledgment latency = **1 ms** During market opening:
- Latency increases to **40 ms**
- TCP retransmissions increase
- Connection establishment becomes slower
- Excessive buffering is observed

The three major TCP-related factors to analyze are:

1. TCP Flow Control
2. Nagle's Algorithm
3. SYN Queue Limitations

These factors can become particularly important when a system must process many small transactions with extremely low latency.

## **Factor 1 — TCP Flow Control**

TCP uses a receiver window (rwnd) to prevent a sender from transmitting more data than the receiver can currently handle.

### **Normal operation** Trading

Server

|  
| Data  
↓

Receiver

|  
| ACK + Window Size  
↓

Trading Server

If the receiving application's buffers become full, the advertised TCP receive window can decrease.

High Order Rate

↓

Receive Buffer Fills

↓

Small TCP Window

↓

Sender Must Slow Down

↓

Order Processing Delay

### **Effect on trading**

During market opening, thousands of orders can arrive rapidly. If the receiver cannot process data quickly enough, TCP flow control causes the sender to reduce its transmission rate. This can result in:

- Delayed order messages
- Delayed acknowledgments
- Increased queueing
- Lower transaction throughput
- Increased application response time

### **Excessive Buffering**

Large buffers can also contribute to bufferbloat. Packets wait in queues instead of being delivered immediately.

Orders

↓↓↓↓↓↓↓  
+-----+  
| Large Buffer |  
| P1 P2 P3 P4... |  
+-----+

↓

Delayed ACK

Thus, simply increasing TCP buffers is not always beneficial for low-latency trading.

### **Factor 2 — Nagle's Algorithm**

Nagle's algorithm is designed to reduce the number of small TCP packets.

When enabled, TCP may temporarily hold small outgoing segments and combine them with subsequent data before transmitting.

#### **Example**

Suppose the trading application sends:

Order 1 → 20 bytes

Order 2 → 25 bytes

Order 3 → 30 bytes

Instead of immediately sending every small segment, Nagle's algorithm may wait for acknowledgment or additional data.

Without Nagle:

Order 1 → SEND

Order 2 → SEND

Order 3 → SEND

With Nagle:

Order 1

↓

Wait

↓

Order 2 + Order 3

↓

SEND

#### **Why this is harmful in high-frequency trading**

For normal applications, reducing the number of packets can improve network efficiency.

For a trading platform, however, **microseconds or milliseconds matter**.

Waiting to combine small messages can increase:

- Order transmission latency
- Order acknowledgment time
- Trade confirmation delay

The observed increase from **1 ms to 40 ms** could therefore be worsened if Nagle's algorithm is interacting with delayed acknowledgments.

#### **Recommended approach**

For latency-sensitive trading connections, use:

#### **TCP\_NODELAY**

This disables Nagle's algorithm and allows small TCP segments to be transmitted immediately.

Trading Application

↓

TCP\_NODELAY

↓  
Immediate TCP transmission  
↓  
Lower latency

### **Factor 3 — SYN Queue Limitations**

TCP establishes connections using the **three-way handshake**:

Client                      Server  
SYN ----->  
  
<----- SYN + ACK  
  
ACK ----->

During market opening, many clients or trading components may establish connections simultaneously.

The server temporarily maintains half-open connections in its **SYN backlog/queue**.

#### **If the queue becomes full**

Thousands of Connection Requests

↓  
SYN Queue  
↓  
+-----+  
| SYN SYN... |  
| SYN SYN... |  
+-----+  
↓  
FULL

↓  
New SYN requests delayed/dropped This can cause:

- Connection establishment delays
- Retransmitted SYN packets
- Failed connections
- Additional handshake latency
- Delayed order processing

### **Effect on trading**

If a trading client needs to establish a new TCP connection before submitting an order, a SYN queue problem directly adds to the transaction latency.

Instead of:

Connection → Order → ACK the  
system experiences:

Connection  
↓  
SYN retransmission

↓  
Connection established

↓  
Order

↓  
ACK

This is especially problematic during market opening when connection requests can arrive in bursts.

### **Comparison of the Three Factors**

| <b>TCP Factor</b> | <b>Main Problem</b>                       | <b>Effect on Trading</b>      |
|-------------------|-------------------------------------------|-------------------------------|
| Flow Control      | Receive window/buffers limit transmission | Delayed order delivery        |
| Nagle's Algorithm | Small packets may be delayed/coalesced    | Higher message latency        |
| SYN Queue         | Too many simultaneous connection attempts | Slow connection establishment |

How These Factors Affect High-Frequency Transaction Processing High-frequency systems require:

- Very low latency
- High throughput
- Predictable response time
- Reliable delivery
- Minimal queueing

### **Flow Control**

When buffers fill, TCP slows the sender.

**Result:** order messages wait longer before reaching the trading engine.

### **Nagle's Algorithm**

Small order messages can be temporarily buffered.

**Result:** individual orders may experience additional transmission delay.

### **SYN Queue Limitations**

New connections may wait or require retransmission.

**Result:** the initial connection setup becomes slower.

Together, these effects can explain why the system moves from approximately **1 ms to 40 ms latency** during peak periods.

### **Recommended TCP Configuration Changes**

#### **A. Disable Nagle's Algorithm Set:**

TCP\_NODELAY = ON

This allows small trading messages to be sent immediately.

### **Benefit**

Order

↓

Immediate TCP transmission

↓

Trading Server

This is particularly appropriate for latency-sensitive order and acknowledgment messages.

## **B. Optimize TCP Receive and Send Buffers**

The system should use appropriately sized TCP buffers rather than simply maximizing them.

Monitor:

- Receive buffer utilization
- Send buffer utilization
- TCP window size
- Queue depth    The objective is:

**Enough buffering to prevent packet loss, but not so much buffering that packets wait excessively.**

Avoid unnecessarily large buffers that can cause bufferbloat.

### **Increase SYN Backlog Capacity**

The server should increase the TCP SYN backlog appropriately to handle connection bursts.

Conceptually:

Small SYN Queue



Queue Full



Dropped/Delayed SYN



Retransmission



Higher Latency

With a properly sized backlog:

Large Connection Burst



SYN Backlog



Connections accepted



Fewer retransmissions

On Linux systems, administrators can tune parameters such as the TCP listen/SYN backlog and related SYN-cookie settings according to the operating system and workload.

### **Use Persistent TCP Connections**

For a high-frequency trading system, it is generally better to **avoid creating a new TCP connection for every order.**

Instead:

TCP Connection



+-- Order 1

+-- Order 2

+-- Order 3

+-- Order 4

+-- Order 5

Rather than:

Order 1 → TCP Handshake

Order 2 → TCP Handshake

Order 3 → TCP Handshake

Persistent connections eliminate repeated three-way handshakes and reduce the pressure on the SYN queue.

### **TCP Retransmission Analysis**

The observed increase in TCP retransmissions should also be investigated.

Possible causes include:

- Network congestion
- Packet loss
- Overloaded network interfaces
- Excessive queues
- Insufficient network capacity

A retransmitted packet must be sent again, increasing latency.

Order



Packet Lost



TCP Retransmission



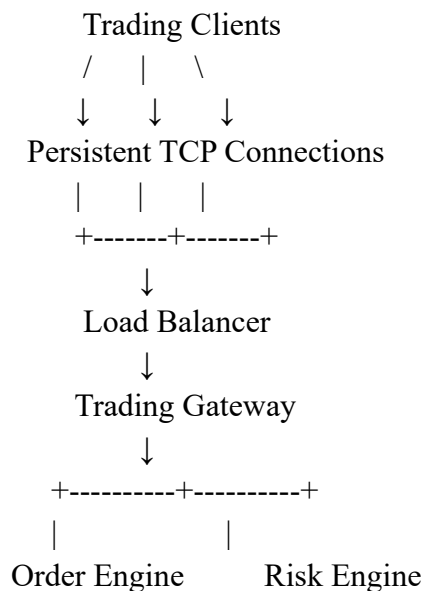
Receiver



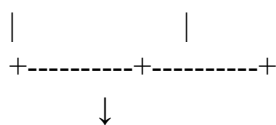
ACK

For a trading platform, even a small amount of packet loss can have a significant effect on tail latency.

### **Recommended Architecture**







Trade Confirmation

TCP configuration should include:

TCP\_NODELAY



Disable unnecessary Nagle delays

Optimized TCP Buffers



Prevent excessive queueing

Adequate SYN Backlog



Handle connection bursts

Persistent Connections



Avoid repeated TCP handshakes

### **Final Recommendation Table**

| <b>Configuration</b>     | <b>Recommended Action</b>               | <b>Expected Benefit</b>          |
|--------------------------|-----------------------------------------|----------------------------------|
| <b>Nagle's Algorithm</b> | Disable using TCP_NODELAY               | Reduces small-message latency    |
| <b>TCP Buffers</b>       | Tune send/receive buffers               | Reduces excessive buffering      |
| <b>SYN Queue</b>         | Increase/tune backlog appropriately     | Handles connection bursts        |
| <b>TCP Connections</b>   | Use persistent connections              | Avoids repeated handshakes       |
| <b>Retransmissions</b>   | Investigate packet loss/congestion      | Improves reliability and latency |
| <b>Monitoring</b>        | Track RTT, retransmissions, queue depth | Detects performance problems     |

### **Conclusion**

The increase from **1 ms to 40 ms** is consistent with multiple TCP-related bottlenecks occurring during the market-opening traffic surge.

The three key factors are:

1. **Flow control** — excessive buffering or a constrained receive window can slow transmission.
2. **Nagle's algorithm** — can delay small trading messages while waiting to combine them.
3. **SYN queue limitations** — can delay new TCP connections when connection requests arrive in large bursts.

**Final Solution**

Disable Nagle's algorithm with TCP\_NODELAY for latency-sensitive connections, tune TCP send/receive buffers to avoid bufferbloat, increase the SYN backlog to handle connection bursts, and use persistent TCP connections wherever possible.

These changes should reduce unnecessary TCP-induced delays while maintaining the reliable, ordered delivery required for financial transactions.